

Programmation

– Examen (1) –

Vendredi, 19 Décembre 2025

Durée: 3 heures

Résumé

L'examen consiste à résoudre un problème en C. Vous disposez de fichiers de tests et d'un squelette de code pour démarrer. À la fin de l'examen, vous devez envoyer votre travail dans une archive `tar.gz` contenant le code source répondant au problème de l'examen à l'adresse email `<emmanuel.fleury@u-bordeaux.fr>`. L'archive devra avoir la forme suivante (remplacez les mots entre `<... >` par votre nom/prénom) :

```
<NOM>_<Prenom>-exam/  
+- naughty-list.c  
+- naughty-list-crt.c  
+- Makefile
```

Vous pouvez laisser vos fichiers de tests dans l'archive. Cela n'a aucune importance pour l'évaluation.

1 Contexte

Hier soir, le lutin en charge des listes des enfants sages a eu un petit accident en manipulant les listes sur son ordinateur. Chaque ligne de ces listes correspond à un enfant et contient une série de nombres entiers représentant les actions de l'enfant durant l'année passée. Ces nombres sont positifs, si c'est une bonne action, et négatifs, si c'est une mauvaise action. De plus, en fonction de l'importance de l'action, le nombre peut être plus ou moins grand. Cela donne donc des séries du genre `"5 - 3 + 17"`, où l'enfant a fait deux bonnes actions (comptées respectivement 5 et 17) et une mauvaise (comptée -3).

Habituellement, le lutin en charge des listes fait la somme de chaque ligne pour décider si l'enfant a été sage ou non. Si la somme est positive, l'enfant est considéré comme sage et il recevra des cadeaux. Sinon, il n'a droit qu'à un morceau de charbon pour le jour de Noël pour lui rappeler qu'il doit rester sage.

Malheureusement, le lutin a malencontreusement supprimé tous les espaces et les signes `"+"` et `"-"` du fichier. Il ne reste donc plus que des séquences d'entiers. Devant cette catastrophe, le Père Noël a décidé de faire appel à vous pour l'aider à décider quels enfants ont été sages l'année passée. Ses instructions sont les suivantes :

« Pour chaque ligne du fichier, essayez de trouver toutes les façons de grouper les entiers en nombres, puis additionnez ou soustrayez ces nombres en insérant des signes `"+"` ou `"-"` entre eux. Parmi toutes les sommes obtenues ainsi, comptez celles qui sont divisibles par 2, 3, 5 ou 7 (on dira que 0 est divisible par tous). Si le résultat final n'est pas nul, alors l'enfant sera considéré comme sage et le contraire sinon. »

De plus, comme cela ne change rien au résultat et que cela permet de réduire le nombre de combinaisons à tester, le Père Noël vous demande de toujours considérer que le signe du premier chiffre est un `"+"`.

Par exemple, si j'ai `'12'` sur une ligne, les combinaisons possibles sont : `'1+2=3'`, `'1-2=-1'` et `'12=12'`. Dans ce cas, 3 et 12 sont divisibles par 3 mais -1 n'est divisible par aucun des diviseurs (2, 3, 5 ou 7). Donc, le résultat sera 2 et on considérera que l'enfant a été sage.

En tout, pour une ligne avec `d` chiffres, il y aura 3^{d-1} façons de combiner les chiffres pour créer autant d'expressions. En effet, entre chaque chiffre, on peut soit insérer un signe `"+"`, soit un signe `"-"`, soit encore ne rien insérer du tout et coller le chiffre avec le suivant.

Tout cela étant dit, le lutin en charge des listes vient vous remettre les fichiers contenant les lignes à analyser. Mais son petit sourire en coin vous laisse penser qu'il y a peut-être anguille sous roche...

2 Spécifications du problème

Les cas à traiter sont donnés sous forme de fichiers texte qui suivront le format suivant :

- **Input** : La première ligne donne le nombre de cas à traiter, N . Chaque cas est donné sous la forme d'une unique chaîne de caractères formée d'une séquence d'entiers décimaux de longueur L .
- **Output** : Pour chaque cas, il faut afficher le numéro du cas ("Case #x: ", où x est le numéro du cas traité) suivi du nombre de cas où le résultat du calcul est divisible par 2, 3, 5 ou 7.
- **Limitations** : $0 \leq N \leq 100$ et $1 \leq L \leq 13$ pour 'medium.in' alors que $1 \leq L \leq 40$ pour 'large.in'.

Input

```
4
1
9
011
12345
```

Output

```
Case #1: 0
Case #2: 1
Case #3: 6
Case #4: 64
```

Au-dessus, vous trouverez un exemple de fichiers d'Input et d'Output. Vous trouverez aussi d'autres fichiers de test dans l'archive de l'examen. Les ensemble de données en entrée sont 'small.in', 'medium.in' et 'large.in'. Et, les réponses attendues sont dans les fichiers ayant les même noms mais avec l'extension '.ans'.

3 Questions

Questions

1. Écrivez le parseur d'arguments qui récupère le nom du fichier d'entrée. En cas de problème, le programme doit afficher un message d'erreur sur `stderr` et retourner `EXIT_FAILURE`. Si le fichier est correctement ouvert, le programme doit renvoyer les solutions sur `stdout`. Voici un exemple d'usage du programme :

```
$> ./naughty-list
naughty-list: error: no input file given!
$> ./naughty-list small.in
Case #1: 0
Case #2: 1
Case #3: 6
Case #4: 64
```

2. Écrivez le parseur de fichiers qui récupère les données des cas à traiter. Nous supposons que le fichier d'entrée est correctement formé, sans erreur de syntaxe, ni oubli de la part de l'utilisateur. Inutile, donc, de passer du temps à essayer d'être robuste lorsque vous parsez le contenu du fichier. Par contre, on s'attend à ce que vous détectiez l'absence du fichier sur les arguments, ou un problème lors de l'ouverture du fichier.

Fonctions à utiliser : `fscanf()`.

3. Écrivez le programme en utilisant une approche par brute-force pour trouver les solutions. La clarté du code, les commentaires et l'efficacité du programme seront pris en compte. Assurez-vous aussi que l'usage de la mémoire soit correct. Tous ces aspects sont importants pour la notation finale.

Le principe de l'énumération par brute-force va consister à créer un tableau `int signs[DIGITS_MAX + 1]` dont chaque cellule peut prendre trois valeurs : 0 (pas de signe, on groupe les chiffres), 1 (signe "+") ou -1 (signe "-"). Comme la première position est toujours positive, on aura toujours `signs[0] = 1`.

Pour énumérer toutes les combinaisons possibles, on va utiliser une boucle `for` avec une variable `mask` qui va aller de 0 à $3^d - 1$ (où d est le nombre de chiffres dans la ligne courante). Cette variable va nous permettre de générer toutes les combinaisons possibles en convertissant la valeur de `mask` en base 3 comme indiqué dans l'algorithme 1, p. 3 en pseudo-code.

Le fait de convertir `mask` en base 3 permet de générer toutes les combinaisons possibles de signes entre les chiffres. En effet, chaque chiffre en base 3 peut prendre trois valeurs (0, 1 ou 2) qui correspondent aux trois possibilités pour chaque position entre les chiffres (rien, "+" ou "-"). On s'assure ainsi de parcourir toutes les combinaisons possibles.

Enfin, il reste à écrire la fonction `evaluate_expression(digits, signs)` qui calcule la somme en fonction du tableau `signs` et la fonction `is_naughty(sum)` qui détermine si la somme est divisible par 2, 3, 5 ou 7.

$3^2 \rightarrow 9$

Algorithm 1 Énumération des combinaisons par brute-force

```

1: for mask de 0 à total_combinations - 1 do
2:   Initialiser signs[0..DIGITS_MAX] à 0
3:   temp ← mask
4:   for i de 0 à len - 2 do
5:     choice ← temp modulo 3
6:     if choice = 0 then
7:       signs[i + 1] ← 0 // Pas de signe
8:     else if choice = 1 then
9:       signs[i + 1] ← +1 // Signe plus
10:    else
11:      signs[i + 1] ← -1 // Signe moins
12:    end if
13:    temp ← temp / 3
14:  end for
15:  sum ← evaluate_expression(digits, signs)
16:  if is_naughty(sum) then
17:    count ← count + 1
18:  end if
19: end for

```

Indication : Vous pouvez commencer par faire imprimer toutes les combinaisons possibles pour une ligne donnée afin de vérifier que votre énumération fonctionne correctement. Ensuite, vous pourrez finir la fonction. Si l'énumération est correcte, il en sera tenu compte dans la notation finale.

4. Si vous avez programmé le brute-force, vous devriez remarquer que le fichier 'medium.in' ($1 \leq L \leq 13$) est traité en un temps raisonnable mais pas le fichier 'large.in' ($1 \leq L \leq 40$). En effet, le nombre de combinaisons possibles croît exponentiellement avec la taille de l'entrée.

Pour traiter ce fichier, il va falloir optimiser le programme. Pour cela, nous allons avoir recours au **théorème des restes chinois**. Celui-ci dit que si on a un ensemble de modulus premiers entre eux, alors on peut travailler modulo le produit de ces modulus sans perdre d'information sur la divisibilité par chacun d'eux. Ici, nos modulus sont 2, 3, 5 et 7 et donc, au lieu de calculer la somme totale de chaque expression, on va calculer la somme modulo 210 (car $210 = 2 \times 3 \times 5 \times 7$). De cette façon, on évite d'avoir des sommes gigantesques à manipuler et on peut se contenter de travailler avec des entiers (int).

Pour implémenter cette approche, on va utiliser une table de programmation dynamique $\text{dyn}[\text{pos}][\text{mod}]$ où pos est la position courante dans la chaîne de chiffres (de 0 à $L-1$) et mod est la valeur de la somme modulo 210 (de 0 à 209). La valeur $\text{dyn}[\text{pos}][\text{mod}]$ contiendra le nombre de façons d'obtenir la valeur mod en utilisant les chiffres de la position 0 à la position pos .

Par exemple, si on prend '12', les valeurs non nulles de ce tableau seront : $\text{dyn}[0][0] = 1$ (cas de base), $\text{dyn}[1][1] = 1$, $\text{dyn}[2][3] = 1$, $\text{dyn}[2][12] = 1$ et $\text{dyn}[2][209] = 1$ (ici -1 modulo 210 = 209).

Ensuite, pour compter le nombre de solutions, il suffit de sommer toutes les valeurs non nulles dont l'index mod est divisible par 2, 3, 5 ou 7. Par exemple, dans le cas de la chaîne '12', on aura : $\text{dyn}[2][3] = 1$ (car 3 est divisible par 3) et $\text{dyn}[2][12] = 1$ (car 12 est divisible par 2 et 3). Donc, le résultat sera 2.

Vous trouverez l'algorithme complet en pseudo-code détaillé dans l'algorithme 2, p. 4.

5. (Bonus) Comparez les complexités des deux approches. Expliquez pourquoi la seconde approche est plus efficace pour traiter le fichier 'large.in'.

Algorithm 2 Comptage des combinaisons avec le théorème des restes chinois

```
1: Initialiser dyn[0..DIGITS_MAX][0..MOD-1] à 0
2: dyn[0][0] ← 1 // cas de base
3: for i de 0 à len - 1 do
4:   for sgn dans {+1, -1} do
5:     cur ← 0
6:     for j de i à len - 1 do
7:       cur ← (cur × 10 + digits[j]) mod MOD
8:       for x de 0 à MOD - 1 do
9:         if dyn[i][x] > 0 then
10:          new_val ← (x + sgn × cur + MOD) mod MOD
11:          dyn[j + 1][new_val] ← dyn[j + 1][new_val] + dyn[i][x]
12:        end if
13:      end for
14:    end for
15:  end for
16: end for
17: result ← 0
18: for x de 0 à MOD - 1 do
19:   if is_naughty(x) then
20:     result ← result + dyn[len][x]
21:   end if
22: end for
23: return result
```
